ENTERPRISE SYSTEM ARCHITECTURE • ENGINEERING PORTFOLIO PIECE

QUEUEFORGE

Distributed Virtual Waiting Room & High-Concurrency Flash Reservation Engine

PROJECT ORIGIN & CONTEXT
Career Level-Up: Junior to Mid/Senior Transition

PRIMARY ARCHITECTURAL PATTERN
Ingress Queuing & In-Memory Distributed Locks

PRODUCTION TECH STACK
Next.js 15, Node/Fastify, Redis, PostgreSQL, WebSockets

HOSTING & TOOLING COST
100% Free Tier ($0.00 / Zero Credit Cards Required)

LOCAL PROJECT WORKSPACE
/home/luffy/Desktop/queueforge

DOCUMENT VERSION & DATE
v1.0 • Master Engineering Blueprint (Oct 2026)

Prepared for: Luffy • Linux Developer Workspace

Google DeepMind Agentic Systems

EXECUTIVE OVERVIEW

Table of Contents & Executive Summary

This master blueprint provides the comprehensive engineering specification for **QueueForge**, a production-grade distributed system designed to handle massive flash-sale traffic spikes without database deadlocks or duplicate inventory sales.

📑 Master Table of Contents

01 **Project Genesis & The "Tutorial Cliché" Problem** Page 3

02 **The Real-World Problem & Commercial Industry Landscape** Page 4

03 **Target Personas & Interactive Seat State Machine** Page 5

04 **End-to-End System Architecture: Request Lifecycle** Page 6

05 **System Architecture Topology & Ingress Patterns** Page 7

06 **Technical Mechanics: Virtual Waiting Room (Redis Sorted Sets)** Page 8

07 **Technical Mechanics: Atomic Distributed Seat Locking (SET NX EX)** Page 9

08 **Database Schema Design (PostgreSQL / Prisma)** Page 10

09 **100% Free-Tier Production Tech Stack & Monorepo Structure** Page 11

10 **Master 8-Week / 4-Phase Execution Roadmap** Page 12

11 **Concurrency Load Testing & Empirical Verification Blueprint** Page 13

12 **Technical Interview Playbook & Portfolio Strategy** Page 14

13 **Quick-Start Guide for /home/luffy/Desktop/queueforge** Page 15

🎯 Why This Blueprint Exists

This document serves as your complete architectural guide, interview playbook, and implementation manual. By building this system inside `/home/luffy/Desktop/queueforge`, you demonstrate mastery over high-concurrency race conditions, distributed locking, in-memory data structures, and asynchronous background worker queues—the exact competencies demanded in senior backend and full-stack engineering roles.

SECTION 01

1. Project Genesis & The "Tutorial Cliché" Problem

1.1 Background: Why Typical Projects Fail in Interviews

Most aspiring software engineers populate their resumes with standard tutorial projects: to-do lists, simple blog engines, basic e-commerce storefronts, or popular Kaggle clones like the *"Plant Disease Detection + Yield Prediction + Weather Recommendation"* bundle.

Hiring managers and senior tech leads reject these projects within seconds for three clear reasons:

- **Zero Concurrency Handling:** These apps assume a single user clicking a button at a time. The moment 50 people click simultaneously, the database deadlocks or crashes.
- **Pure Synchronous CRUD:** The server only operates when a user makes an HTTP request. There are no background workers, no distributed caches, and no event-driven mechanisms.
- **Tutorial Cliché:** Recruiters review dozens of identical Kaggle/YouTube clones every week. They do not demonstrate deep engineering ability or systems design comprehension.

> 💡 **The Antidote: Real-World Enterprise System Design**
> **QueueForge** is specifically engineered to be the exact opposite of a generic clone. It replicates the mission-critical infrastructure deployed by companies like **Ticketmaster, Shopify, and Cloudflare** to survive sudden traffic surges (e.g., Coldplay ticket drops, sneaker drops, university course enrollments) without failing or double-booking inventory.

1.2 What QueueForge Is: The Elevator Pitch

QueueForge is a self-hosted, full-stack distributed virtual waiting room and high-concurrency seat reservation engine. When traffic spikes beyond normal operational capacity:

1. It intercepts excess traffic before it touches the primary SQL database, queuing users into a high-throughput **Redis FIFO Sorted Set**.
2. It streams live queue positions (*"You are #142 in line"*) to the client using **WebSockets / Server-Sent Events**.
3. It meters admitted users with cryptographically signed, short-lived **JWT passes**.
4. It provides an interactive visual SVG seat map where inventory reservations are protected by **atomic distributed Redis locks** (`SET NX EX`) with 10-minute hold timers and automatic background expiration sweepers.
5. It guarantees idempotent checkout via PostgreSQL ACID transactions, ensuring zero duplicate bookings or double credit card charges under heavy write contention.

SECTION 02

2. The Real-World Problem & Market Landscape

2.1 The Disaster of High-Demand Flash Sales

Imagine a concert ticket drop: **50,000 enthusiastic fans visit a ticketing page at exactly 12:00:00 PM for 500 available seats**.

❌ Naive Architecture (Junior Dev)

- 50,000 HTTP requests hit the database directly.
- Database connection pool (usually ~20-50 connections) is instantly exhausted.
- Queries stall, response times spike past 30s, and the server crashes with `504 Gateway Timeout`.
- **Race Conditions:** Multiple users query `SELECT WHERE is_reserved=false` and buy the same seat at the same second.

✅ QueueForge Architecture (Mid/Senior)

- Database is completely shielded behind an in-memory Redis gatekeeper.
- Users wait comfortably in a virtual queue with live countdown telemetry.
- Admissions are metered steadily (e.g., 25 users / 10s) based on server processing capacity.
- Atomic in-memory distributed locks guarantee that a seat can only be held by exactly 1 user. Zero double-bookings.

2.2 Existing Industry Solutions

This exact problem is solved by multi-million-dollar enterprise software suites. Building your own working implementation demonstrates senior-level system design literacy:

EXISTING SOLUTION	PROVIDER / USAGE	ARCHITECTURE & COMMERCIAL COST
Queue-it	Global market leader in virtual waiting rooms (Ticketmaster, major airlines).	Proprietary SaaS. Ingress edge redirection. Starts at **$15,000 – $50,000+ per year**.
Cloudflare Waiting Room	Edge queuing integrated directly into Cloudflare CDN/DNS.	Queues excess requests at edge points of presence. Requires Enterprise plan ($100s–$1,000s/mo).
Shopify Checkout Queue	Proprietary internal infrastructure for Black Friday / Cyber Monday.	Closed-source custom architecture handling millions of checkouts per minute.
AWS Virtual Waiting Room	AWS open-source reference architecture for cloud engineers.	Multi-service architecture (API Gateway + DynamoDB + Lambda + CloudFront + IoT Core).

🏆 Why QueueForge Beats All of Them as a Portfolio Piece

Enterprise platforms like *Queue-it* are closed-source black boxes costing tens of thousands of dollars. **QueueForge** combines both the **waiting room** AND the **live visual seat map** with **distributed locks** into a clean, 100% free-tier, self-hosted monorepo with automated load-testing verification.

SECTION 03

3. Target Personas & Interactive Seat State Machine

👤 Persona 1: The Ticket Buyer (End User)

Fans attempting to purchase tickets during sudden surges.

- Enters waiting room and sees animated position indicator and real-time wait estimate.
- Redirected automatically when admitted; receives a 15-minute cryptographically signed JWT pass.
- Interacts with dynamic SVG seat map (zooming, selecting seats).
- Gets an instant 10-minute hold with a visual countdown timer.
- Completes idempotent checkout before the hold expires.

🎪 Persona 2: The Event Organizer

Event promoters, theater managers, conference hosts.

- Creates events with start/end times and seat tier pricing.
- Generates venue seating layouts (sections, rows, seats).
- Monitors live sales telemetry: percentage of seats sold, held, and available.
- Configures max inventory purchase limits per user.

⚙️ Persona 3: SRE & Platform Admin

Site reliability engineers and system operators maintaining uptime.

- Adjusts **Drain Rate Controller** dynamically (e.g. throttle to 15 users/10s during DB load spikes).
- Monitors Redis lock contention rates and BullMQ background worker queue depths.
- Executes emergency resets or manual seat releases if necessary.

3.1 Interactive Seat State Machine

To prevent confusion and eliminate race conditions, each seat moves through a strictly enforced state machine across both client and server:

🟢 **AVAILABLE** 🟠 **HELD (Other User)** 🔵 **SELECTED (You - 10m Hold)** 🔴 **SOLD / CONFIRMED**

CURRENT STATE	TRIGGER / EVENT	NEXT STATE	SYSTEM ACTION
AVAILABLE	User clicks seat	HELD (You)	Redis `SET NX EX 600` succeeds; WebSocket emits to other clients as HELD.
AVAILABLE	Other user clicks seat	HELD (Other)	Lock acquired by another client; updates to yellow on your screen.
HELD	Another user attempts click	HELD (No Change)	Instant HTTP 409 Conflict. Redis lock rejects without touching database.
HELD (You)	Timer reaches 0 / Abandoned	AVAILABLE	BullMQ worker sweeps key; Redis lock released; WebSocket turns seat green for all.
HELD (You)	Payment confirmed	SOLD	PostgreSQL transaction marks seat sold; Redis lock deleted; WebSocket turns seat red.

SECTION 04

4. End-to-End System Architecture: Request Lifecycle

When 50,000 requests hit the system during a ticket drop, QueueForge guides each request through a disciplined, six-step architectural pipeline:

1. **Traffic Ingress & Threshold Check:** Fastify/Express edge middleware monitors active buyer sessions in Redis. If active checkout sessions exceed capacity (e.g. > 500), incoming users are redirected immediately to the Virtual Waiting Room.

2. **Virtual Waiting Room Placement:** User receives a persistent UUID cookie and is added to an in-memory Redis Sorted Set via `ZADD queue:event_101 <timestamp> <uuid>`. Rank is determined in sub-millisecond O(log N) time via `ZRANK`.

3. **Real-Time Waiting Telemetry:** An open WebSocket or Server-Sent Events (SSE) connection feeds live rank and estimated wait times to the user: *"You are #84 in line. Estimated wait: 28 seconds."*

4. **Queue Drain & Cryptographic Admission Pass:** An autonomous background worker pops 25 users every 10 seconds via `ZPOPMIN`. Admitted users are issued a signed 15-minute JWT Queue Pass granting access to the seat reservation API.

5. **Interactive Seat Map & Atomic Reservation:** User clicks Seat `B-14`. Server runs atomic Redis command: `SET lock:seat:B14 <userId> NX EX 600`.
   - If returns `OK`: Hold secured for 10 minutes. Emits WebSocket broadcast turning seat yellow for everyone else.
   - If returns `null`: Reject instantly with `409 Conflict: Seat already held`. Database is completely unburdened.

6. **Checkout Settlement or Auto-Sweep:**
   - **Success:** User submits payment with `Idempotency-Key`. Server opens PostgreSQL transaction: validates Redis lock, inserts order record, updates `seats.status = 'SOLD'`, deletes Redis lock, and broadcasts red status.
   - **Timeout / Abandon:** User closes tab or 10-minute timer expires. BullMQ delayed job releases Redis key and broadcasts green status.

SECTION 04 (CONTINUED)

5. System Architecture Topology & Ingress Patterns

5.1 Architectural Topology Diagram

The system strictly separates in-memory transient traffic (locks, queue sorting, WebSocket mesh) from persistent ACID transactions (orders, event catalogs):



5.2 Key Ingress Principles

- **Edge Admission Gating:** No unadmitted request can reach database-touching routes. The JWT Queue Pass acts as a cryptographically signed gate pass.
- **In-Memory Contention:** High-frequency seat clicks never execute SQL queries. All reservation contests are decided in Redis in under 2ms.
- **Event-Driven State Push:** Seat state changes are pushed through WebSockets rather than polled via HTTP, minimizing server overhead.

SECTION 05 (PART 1)

6. Technical Mechanics: Virtual Waiting Room (Redis Sorted Sets)

6.1 Mathematical Principles of In-Memory Queuing

Standard relational queues suffer severe slowdown under concurrency because `SELECT ... FOR UPDATE` locks table rows and degrades to O(N) sequential scans. In contrast, Redis Sorted Sets use a skip-list and hash-map combination guaranteeing **O(log N)** insertion and lookup performance.

6.2 Implementation: QueueService

```
// apps/api/src/services/queue.service.ts
import { redis } from "../lib/redis";

export class QueueService {
  private queueKey(eventId: string): string {
    return `queue:event:${eventId}`;
  }

  // 1. Enqueue User: Uses current Unix timestamp as score
  async enterQueue(eventId: string, userId: string): Promise<{ rank: number; waitTimeSeconds: number }> {
    const key = this.queueKey(eventId);
    const score = Date.now(); // Guarantees strict FIFO ordering

    // ZADD is O(log N)
    await redis.zadd(key, score, userId);

    const rank = await this.getRank(eventId, userId);
    // Based on an admission rate of 25 users per 10 seconds (2.5 users/sec)
    const waitTimeSeconds = Math.ceil(rank / 2.5);

    return { rank, waitTimeSeconds };
  }

  // 2. Fetch Live Rank: ZRANK returns 0-indexed position
  async getRank(eventId: string, userId: string): Promise {
    const key = this.queueKey(eventId);
    const rank = await redis.zrank(key, userId);
    return rank !== null ? rank + 1 : -1;
  }

  // 3. Queue Drain Worker: Atomically pops the earliest N users
  async admitBatch(eventId: string, batchSize: number = 25): Promise {
    const key = this.queueKey(eventId);
    // ZPOPMIN atomically extracts the lowest scores
    const popped = await redis.zpopmin(key, batchSize);
    // Filter array to extract member IDs (every even index)
    return popped.filter((_, idx) => idx % 2 === 0);
  }
}
```

> 💡 **Why Skip Lists Outperform B-Trees for Flash Sales**
> Redis Sorted Sets maintain a probabilistic skip-list in RAM. Even with 100,000 waiting users, evaluating `ZRANK` takes less than 1 millisecond and requires only ~12MB of RAM, shielding the database completely.

SECTION 05 (PART 2)

7. Technical Mechanics: Atomic Distributed Seat Locking (SET NX EX)

7.1 Eliminating Double-Bookings with Atomic In-Memory Locks

When two users click Seat `B-14` at the exact same millisecond, naive systems suffer read-modify-write race conditions. QueueForge solves this via Redis atomic commands with zero locking on SQL tables.

7.2 Implementation: LockService

```
// apps/api/src/services/lock.service.ts
import { redis } from "../lib/redis";

export class LockService {
  private lockKey(seatId: string): string {
    return `lock:seat:${seatId}`;
  }

  // Atomically acquire a 10-minute hold (600 seconds)
  async acquireSeatHold(seatId: string, userId: string, ttlSeconds: number = 600): Promise {
    const key = this.lockKey(seatId);

    // NX = Only set key if it does NOT exist (Atomicity guarantee)
    // EX = Automatically expire key after ttlSeconds (Auto-release protection)
    const result = await redis.set(key, userId, "NX", "EX", ttlSeconds);

    return result === "OK";
  }

  // Atomically release hold using Lua Script
  // Guarantees that User A cannot accidentally release a lock held by User B
  async releaseSeatHold(seatId: string, userId: string): Promise {
    const key = this.lockKey(seatId);
    const luaScript = `
      if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("del", KEYS[1])
      else
        return 0
      end
    `;
    const result = await redis.eval(luaScript, 1, key, userId);
    return result === 1;
  }
}
```

> 🔒 **The Ownership-Safe Lua Release Pattern**
> Notice the Lua script in `releaseSeatHold`. If User A's hold expired at 10:00:00 and User B acquired it at 10:00:01, a naive `DEL` from User A would wrongfully steal User B's lock. The Lua script evaluates equality and deletion atomically inside Redis, preventing lock-stealing bugs.

SECTION 06

8. Database Schema Design (PostgreSQL / Prisma)

Relational data is stored in PostgreSQL and managed with Prisma ORM. The relational database is strictly protected from high-frequency locking, handling only durable entity state and completed transactions.

```
// Core Catalogs: Events & Venues
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum EventStatus {
  UPCOMING
  ON_SALE
  SOLD_OUT
  COMPLETED
}

model Event {
  id                String      @id @default(uuid
())
  title             String
  venueId           String
  venue             Venue       @relation(fields:
[venueId], references: [id])
  eventDate         DateTime
  status            EventStatus @default(UPCOMING)
  maxTicketsPerUser Int         @default(4)
  seats             Seat[]
  orders            Order[]
  createdAt         DateTime    @default(now())
}

model Venue {
  id        String   @id @default(uuid())
  name      String
  city      String
  totalRows Int
  totalCols Int
  events    Event[]
}
```

```
// Inventory & Transaction Settlement
enum SeatStatus  { AVAILABLE, SOLD, BLOCKED }
enum OrderStatus { PENDING, COMPLETED, CANCELLED }

model Seat {
  id         String     @id @default(uuid())
  eventId    String
  event      Event      @relation(fields: [eventI
d], references: [id])
  section    String     // e.g., "VIP", "Floor"
  row        String     // e.g., "A", "B"
  number     Int        // e.g., 1, 2, 3
  priceCents Int        // e.g., 7500 ($75.00)
  status     SeatStatus @default(AVAILABLE)
  orderId    String?
  order      Order?     @relation(fields: [orderI
d], references: [id])
  updatedAt  DateTime   @updatedAt

  @@unique([eventId, section, row, number])
  @@index([eventId, status])
}

model Order {
  id               String      @id @default(uuid
())
  eventId          String
  event            Event       @relation(fields:
[eventId], references: [id])
  userId           String
  totalAmountCents Int
  status           OrderStatus @default(PENDING)
  idempotencyKey   String      @unique
  seats            Seat[]
  createdAt        DateTime    @default(now())
}
```

> ⚠️ **Idempotency Key Pattern: Preventing Double Credit Card Charges**
>
> Notice the unique index on `idempotencyKey` in the `Order` table. When users repeatedly click the "Confirm Purchase" button, the client sends a unique UUID in the `Idempotency-Key` header. PostgreSQL enforces unique constraints, ensuring identical requests return cached receipts rather than creating duplicate orders.

SECTION 07

9. 100% Free-Tier Production Tech Stack & Monorepo

QueueForge can be completely developed locally with Docker and deployed to production for **$0.00** with zero credit card requirements:

LAYER	SELECTED TECHNOLOGY	PRODUCTION FREE TIER	WHY CHOSEN FOR QUEUEFORGE
Frontend	Next.js 15 (App Router), Tailwind CSS, Framer Motion	**Vercel Free Tier**	SSR for initial page loads; client-side interactive SVG seat grid and real-time WebSocket listeners.
Backend API	Node.js TypeScript + Fastify / Express	**Render.com Free Web Service**	Fastify handles high HTTP throughput and WebSockets with significantly lower memory footprint.
Cache & Lock	Redis (v7.2)	**Upstash Redis (Free Tier)** / Docker	Sub-millisecond `ZADD/ZRANK` waiting rooms and atomic `SET NX EX` distributed locks.
Database	PostgreSQL (v16) with Prisma ORM	**Supabase Free Tier**	ACID transactional integrity for completed tickets and strict uniqueness constraints on idempotency keys.
Real-Time	WebSockets (`ws` / `Socket.io`)	Direct API WebSockets on Render	Instant seat color synchronization across all connected clients without polling overhead.
Background Jobs	BullMQ / pg-boss	Worker process on Render / Docker	Handles scheduled queue drain batches and automatic sweeps of expired 10-minute holds.
Load Testing	k6 / Autocannon	Local CLI & GitHub Actions	Allows writing repeatable stress tests simulating 1,000 concurrent users to prove zero double-bookings.

9.1 Monorepo Workspace Directory Tree

```
/home/luffy/Desktop/queueforge/
├── apps/
│   ├── web/                    # Next.js 15 Frontend (App Router, SVG Seat Grid, Waiting Room UI)
│   ├── api/                    # Fastify Backend API (QueueService, LockService, WebSockets)
│   └── worker/                 # Background Job Processor (BullMQ Drain & Sweeper Jobs)
├── packages/
│   ├── database/               # Prisma Schema, Migrations, Venue Seeders
│   ├── types/                  # Shared TypeScript Interfaces & WebSocket Event DTOs
│   └── load-tests/             # k6 Concurrency Stress Scripts (200-1,000 virtual users)
├── docker-compose.yml          # Local Postgres (5432) & Redis (6379)
└── README.md                   # Architecture Diagrams, Benchmarks & Concurrency Proofs
```

SECTION 08

10. Master 8-Week / 4-Phase Execution Roadmap

P1

PHASE 1: Project Skeleton & Database Foundations (Weeks 1 & 2)

- Initialize npm workspaces / Turborepo in `/home/luffy/Desktop/queueforge`.
- Set up `docker-compose.yml` for local PostgreSQL & Redis containers.
- Model Prisma database schema (Events, Venues, Seats, Orders) and run initial migrations.
- Create a seeder script generating a realistic concert venue layout with 500 interactive seats.

P2

PHASE 2: Concurrency & Lock Engine - The Core Backend (Weeks 3 & 4)

- Implement `LockService` in Fastify using atomic Redis `SET NX EX 600`.
- Write Lua script for safe, ownership-verified hold releases.
- Implement BullMQ worker to sweep expired holds and return seats to inventory.
- **Automated Stress Test:** Write a Vitest/k6 script sending 50 simultaneous HTTP requests for Seat `A-1`; assert exactly 1 success and 49 rejections.

P3

PHASE 3: Virtual Waiting Room & WebSockets (Weeks 5 & 6)

- Implement Redis Sorted Set queue (`ZADD`, `ZRANK`, `ZCARD`).
- Build Queue Drain Worker: periodically pops batches of users and issues signed JWT passes.
- Configure WebSocket server broadcasting seat status updates across connected clients.
- Enforce middleware: users cannot reach the seat reservation API without an active JWT pass.

P4

PHASE 4: Frontend UI, Load Testing & Cloud Deployment (Weeks 7 & 8)

- Build zoomable, pannable SVG Seat Grid component in Next.js with real-time color states.
- Build Waiting Room screen with animated progress bar and audio chime upon admission.
- Implement idempotent checkout form with 10-minute hold countdown timer.
- Run high-concurrency load test with **k6** (1,000 simulated users) and generate metrics charts.
- Deploy frontend to Vercel and backend/worker to Render.com (100% free tier).

📅 **Timeline Flexibility**

If you are dedicating full-time hours (20-30 hrs/week), this roadmap can comfortably be compressed into **3 to 4 weeks** by executing Phase 1 and 2 in Week 1, Phase 3 in Week 2, and Phase 4 in Week 3.

SECTION 09

11. Concurrency Load Testing & Verification Blueprint

The core value proposition of QueueForge on a developer portfolio is **empirical verification**. Anyone can write code and claim it is concurrent; you will provide repeatable scripts that prove it.

11.1 The k6 High-Concurrency Stress Test Script

```
// packages/load-tests/seat-contention.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  // Simulate 200 concurrent users attacking the exact same seat within 5 seconds
  vus: 200,
  duration: '5s',
};

export default function () {
  const url = 'http://localhost:3001/api/seats/A-12/hold';
  const payload = JSON.stringify({
    userId: `user-${__VU}-${__ITER}`,
    eventId: 'event-coldplay-2026',
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer test-jwt-pass',
    },
  };

  const res = http.post(url, payload, params);

  // We expect EXACTLY ONE 200 OK, and ALL OTHER 199 responses to be 409 Conflict
  check(res, {
    'status is 200 or 409': (r) => r.status === 200 || r.status === 409,
    'not a server crash 500': (r) => r.status !== 500,
  });

  sleep(0.1);
}
```

11.2 Verification Metrics to Showcase in Your GitHub README

📊 Latency & Throughput KPIs

- **P95 Response Time:** < 15ms under 1,000 concurrent requests.
- **Lock Acquisition Time:** Sub-2ms Redis in-memory operation.
- **Database Load:** 0 direct queries during high-concurrency contention phase.
- **Error Rate:** 0% server crashes (500s); graceful 409 Conflict handling.

🛡️ Data Integrity Guarantee

- **Double Bookings:** Exactly 0 across 10,000 simulated purchases.
- **Lock Leaks:** 0 stranded holds; 100% of abandoned reservations reclaimed within 600s.
- **Idempotency Safety:** Duplicate button clicks produce identical transaction receipts without duplicate billing.

SECTION 10

12. Technical Interview Playbook & Portfolio Pitch

12.1 The Interview Script: "Tell Me About a Complex Project You Built"

When an interviewer asks you about a complex project, do **not** say: *"I built a ticket selling website with Next.js and Postgres."* Instead, deliver this senior-level system design response:

> 🎙️ **Your Interview Pitch Script**
>
> *"I engineered **QueueForge**, a distributed high-concurrency reservation engine designed to solve the traffic spike and race condition problems common during flash sales like Ticketmaster concert drops.*
>
> *The biggest architectural hurdle was preventing database deadlocks and double-bookings when tens of thousands of users attempt to purchase the exact same seat simultaneously. To solve this, I designed a multi-layer ingress system: incoming users are metered through an in-memory **Redis Sorted Set virtual waiting room** that streams live position telemetry via WebSockets and issues temporary cryptographic access passes.*
>
> *Once inside the interactive seat map, seat holds are managed using **atomic Redis distributed locks** with a 10-minute TTL and ownership-verified Lua script releases. This completely isolates our PostgreSQL database from write thrashing. I verified the concurrency model by executing automated **k6 load tests** simulating 1,000 concurrent users attacking identical inventory, achieving sub-15ms P95 response times with zero deadlocks and zero duplicate bookings."*

12.2 Top 4 System Design Questions This Project Answers

SYSTEM DESIGN QUESTION	HOW QUEUEFORGE DIRECTLY DEMONSTRATES MASTERY
"How do you prevent race conditions without table-locking the database?"	Explain the atomic `SET NX EX` pattern in Redis. Show how in-memory locks return in <2ms, rejecting competing threads immediately without locking Postgres rows.
"How do you handle fair queuing under extreme traffic surges?"	Explain the Redis Sorted Set (`ZADD` / `ZRANK`) data structure, timestamp scoring for strict FIFO fairness, and background batch drain controllers.
"How do you clean up abandoned reservations if a user closes their tab?"	Explain TTL self-destruct mechanisms and BullMQ delayed job schedulers that broadcast WebSocket release events to repopulate the seat canvas.
"How do you prevent double payment charges if a user clicks submit twice?"	Explain the client-generated `Idempotency-Key` pattern and PostgreSQL unique constraint enforcement within ACID transactions.

SECTION 11

13. Quick-Start Guide for /home/luffy/Desktop/queueforge

13.1 Immediate Initialization Steps

The project directory is already prepared at `/home/luffy/Desktop/queueforge`. Follow these exact steps to begin Phase 1:

```
# Step 1: Navigate to the project workspace
cd /home/luffy/Desktop/queueforge

# Step 2: Initialize root package.json for workspaces
npm init -y

# Step 3: Create monorepo directory tree
mkdir -p apps/web apps/api apps/worker packages/database packages/types packages/load-tests
```

13.2 Local Docker Compose Environment (PostgreSQL + Redis)

Create `docker-compose.yml` in the root of `queueforge` to run both PostgreSQL and Redis locally with a single command:

```
version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    container_name: queueforge-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgrespassword
      POSTGRES_DB: queueforge
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7.2-alpine
    container_name: queueforge-redis
    ports:
      - "6379:6379"
    volumes:
      - redisdata:/data

volumes:
  pgdata:
  redisdata:
```

```
# Start local database and Redis services
docker compose up -d
```

> 🚀 **You Are Ready to Build!**
> You now have the complete engineering blueprint, architectural foundation, and step-by-step roadmap for **QueueForge**. Whenever you are ready to write the first lines of code, simply open the project and let's implement Phase 1!